



<PCAPProgramming>

📅 작성일	2026년 7월 1일
📁 과목명	네트워크보안
🌟 과제	완료
≡ 이름	4기 39반 박성현

📄 목차

본 보고서는 PCAP API를 활용하여 네트워크 패킷을 캡처하고, Ethernet / IP / TCP 계층 정보를 분석하여 HTTP 메시지를 출력하는 프로그램을 구현하는 것을 목표로 한다.

본 과제는 제공된 `sniff_improved.c` 와 `myheader.h` 를 기반으로 구현을 진행하였다.

`sniff_improved.c` 의 패킷 캡처 및 기본적인 헤더 분석 구조를 시작점으로 사용하였으며, 과제 요구사항에 맞게 Ethernet Header, IP Header, TCP Header의 정보를 출력하도록 수정하였다.

또한 TCP Header 이후의 Payload 위치를 계산하여 HTTP Message(Application Layer Data)를 출력하는 기능을 추가하였다.

구성은 다음과 같다.

1. 실습 환경 구성
2. Ethernet Header: src MAC / dst MAC
3. IP Header: src IP / dst IP
4. TCP Header: src port / dst port
5. HTTP Message 출력 (Application Layer Data)
6. 실행 결과
7. 개선 사항

📄 1. 실습 환경 구성

본 실습은 Linux 환경에서 진행되었으며, C 언어와 libpcap 라이브러리를 이용하여 패킷 캡처 프로그램을 구현하였다.

네트워크 인터페이스는 ens33을 사용하였으며, 패킷 캡처 대상 트래픽 생성을 위해 netcat(nc) 명령어를 이용하였다.

실습 전 네트워크 환경 확인을 위해 ifconfig 명령어를 사용하였다.

```
psh@psh-VMware-Virtual-Platform:~/바탕화면/network_security_codes/Sniffing_Spoof
\ng/C_sniff$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.196.132 netmask 255.255.255.0 broadcast 192.168.196.255
    inet6 fe80::20c:29ff:fe16:1385 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:16:13:85 txqueuelen 1000 (Ethernet)
    RX packets 41258 bytes 48434680 (48.4 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 10712 bytes 2978398 (2.9 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

```
Ethernet adapter VMware Network Adapter VMnet8:

Connection-specific DNS Suffix . : 
Link-local IPv6 Address . . . . . : fe80::8702:bfd9:d524:ccb4%9
IPv4 Address. . . . . : 192.168.196.1
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . :
```

- 인터페이스: ens33
- IP 주소: 192.168.196.132
- 상태: UP

📌 TCP 통신 테스트 환경

TCP 패킷 생성을 위해 다음과 같이 nc 명령어를 사용하였다.

1) 서버 실행 (수신 대기)

```
nc -l -p 9000
```

2) 클라이언트 실행 (데이터 전송)

```
nc 192.168.196.132 9000
```

위 명령을 통해 로컬 네트워크 상에서 TCP 연결을 생성하고, 입력 데이터를 전송하여 패킷이 발생하도록 구성하였다.

📌 패킷 구조

네트워크에서 데이터가 전송될 때는 응용 계층에서 생성된 데이터에 각 계층의 헤더가 순차적으로 추가되는 **캡슐화(Encapsulation)** 과정을 거친다. 따라서 하나의 패킷은 Ethernet Header, IP Header, TCP Header, 그리고 Application Layer의 데이터(HTTP Message)로 구성된다.

이번 과제에서는 패킷의 시작 위치부터 각 Header의 길이를 이용하여 Ethernet Header, IP Header, TCP Header를 순서대로 분석한 후, 마지막으로 HTTP Message를 출력하도록 구현하였다.

```
+-----+-----+-----+-----+
| Ethernet Header | IP Header | TCP Header | HTTP Message |
+-----+-----+-----+-----+
```

실습 환경 -> TCP 통신 테스트 -> 패킷 구조 (캡슐화) -> Ethernet Header -> IP Header -> TCP Header -> HTTP Message

📄 2. Ethernet Header: src MAC / dst MAC

수신된 패킷은 Ethernet Header는 네트워크 계층보다 하위 계층으로, 송신자와 수신자의 MAC 주소 정보를 포함한다.

패킷의 시작 주소를 `ethheader` 구조체로 변환한 후 Source MAC과 Destination MAC을 출력하도록 구현하였다.

또한 EtherType을 확인하여 IPv4 패킷인 경우에만 이후의 Header를 분석하도록 하였다.

```
struct ethheader *eth = (struct ethheader *)packet;
```

EtherType 값을 통해 IPv4 패킷인지 확인하였으며, 0x0800인 경우 IP 패킷으로 처리하였다.

```
if (ntohs(eth->ether_type) == 0x0800)
```

출력 결과로 Source MAC과 Destination MAC을 확인할 수 있다.

```
//MAC 주소를 출력하는 함수
printf("    MAC Address: %02x:%02x:%02x:%02x:%02x:%02x\n",
    eth->ether_shost[0], eth->ether_shost[1], eth->ether_shost[2],
    eth->ether_shost[3], eth->ether_shost[4], eth->ether_shost[5]);
printf("    MAC Address: %02x:%02x:%02x:%02x:%02x:%02x\n",
    eth->ether_dhost[0], eth->ether_dhost[1], eth->ether_dhost[2],
    eth->ether_dhost[3], eth->ether_dhost[4], eth->ether_dhost[5]);
```

3. IP Header: src IP / dst IP

IP Header에서는 출발지와 목적지 IP 주소를 출력하였다.

또한 `iph_protocol` 값을 이용하여 상위 프로토콜을 확인하고, TCP 패킷인 경우에만 이후의 TCP Header를 분석하도록 구현하였다.

IP Header의 길이는 옵션 필드의 존재 여부에 따라 달라질 수 있으므로 `iph_ihl` 값을 이용하여 TCP Header의 시작 위치를 계산하였다.

```
struct ipheader * ip = (struct ipheader *) (packet + sizeof(struct ethheader));
```

해당 구조체를 통해 다음 정보를 출력하였다.

- Source IP Address
- Destination IP Address

```
printf("      From: %s\n", inet_ntoa(ip->iph_sourceip));
printf("      To: %s\n", inet_ntoa(ip->iph_destip));
```

또한 상위 프로토콜을 확인하기 위해 `iph_protocol` 값을 사용하였다.

```
switch(ip->iph_protocol)
```

4. TCP Header: src port / dst port

TCP Header는 IP Header 뒤에 위치하지만, IP Header의 길이가 고정되어 있지 않으므로 `iph_ihl` 값을 이용하여 TCP Header의 시작 위치를 계산하였다.

이후 Source Port와 Destination Port를 출력하여 송신 포트와 수신 포트를 확인하였다.

```
struct tcpheader *tcp = (struct tcpheader *) (packet + sizeof(struct ethheader) + (ip->iph_ihl * 4));
```

출력 정보는 다음과 같다.

- Source Port
- Destination Port

```
printf("  Source Port: %d\n", ntohs(tcp->tcp_sport));
printf("  Destination Port: %d\n", ntohs(tcp->tcp_dport));
```

IP header length(`iph_ihl`) 값을 이용하여 정확한 TCP 시작 위치를 계산하였다.

5. HTTP Message 출력 (Application Layer Data)

HTTP Message는 TCP Header 이후의 Payload 영역에 존재한다.

TCP Header 역시 옵션의 유무에 따라 길이가 달라질 수 있으므로 `TH_OFF()` 를 이용하여 Payload의 시작 위치를 계산하였다.

또한 IP Header의 Total Length에서 IP Header와 TCP Header의 길이를 제외하여 Payload의 크기를 계산하였다.

```
u_char *http = (u_char *) (packet + sizeof(struct ethheader) + (ip->iph_ihl * 4) + TH_OFF(tcp) * 4);
```

payload 길이는 IP total length에서 header 크기를 제외하여 계산하였다.

```
int payload_len = ntohs(ip->iph_len) - (ip->iph_ihl * 4) - (TH_OFF(tcp) * 4);
```

출력 시에는 가독성을 위해 개행 문자(`\r`, `\n`)를 '.'으로 변환하였다.

```
printf("  httpmessage: ");
for (int i = 0; i < payload_len; i++) {
    unsigned char c = http[i];
    if (c == '\r' || c == '\n') c = '.';
    putchar(c);
}
```

6. 실행 결과

프로그램 실행 시 Ethernet, IP, TCP 정보와 함께 HTTP payload가 출력된다.

예시 출력은 다음과 같다.

nc 클라이언트에서 "안녕하세요"를 입력하여 전송한 결과, 프로그램에서 Source/Destination MAC, IP, Port 정보와 함께 Payload 영역에 전송한 문자열이 출력되는 것을 확인하였다.

```
ling/C_sniff$ gcc -o sniff_improved sniff_improved.c -lpcap
psh@psh-VMware-Virtual-Platform:~/바탕 화면/network_security_codes/Sniffing_Spoof
ling/C_sniff$ sudo ./sniff_improved
[sudo] psh 암호:
MAC Address: 00:50:56:c0:00:08
MAC Address: 00:0c:29:16:13:85
From: 192.168.196.1
To: 192.168.196.132
Protocol: TCP
Source Port: 5604
Destination Port: 9000
httpmessage: 안녕하세요 .
MAC Address: 00:0c:29:16:13:85
MAC Address: 00:50:56:c0:00:08
From: 192.168.196.132
To: 192.168.196.1
Protocol: TCP
Source Port: 9000
Destination Port: 5604
httpmessage:
```

깃허브 링크

https://github.com/tjdgus2670/WHs_subject/blob/main/%EB%84%A4%ED%8A%B8%EC%9B%8C%ED%81%AC%EB%B3%B4%EC

전체코드

```
#include <stdlib.h>
#include <stdio.h>
#include <pcap.h>
#include <arpa/inet.h>

/* Ethernet header */
struct ethheader {
    u_char ether_dhost[6]; /* destination host address */
    u_char ether_shost[6]; /* source host address */
    u_short ether_type; /* protocol type (IP, ARP, RARP, etc) */
};

/* IP Header */
struct ipheader {
    unsigned char iph_ihl:4, //IP header length
    iph_ver:4; //IP version
    unsigned char iph_tos; //Type of service
    unsigned short int iph_len; //IP Packet length (data + header)
    unsigned short int iph_ident; //Identification
    unsigned short int iph_flag:3, //Fragmentation flags
    iph_offset:13; //Flags offset
    unsigned char iph_ttl; //Time to Live
    unsigned char iph_protocol; //Protocol type
    unsigned short int iph_checksum; //IP datagram checksum
    struct in_addr iph_sourceip; //Source IP address
    struct in_addr iph_destip; //Destination IP address
};

/* TCP Header */
struct tcpheader {
    u_short tcp_sport; /* source port */
    u_short tcp_dport; /* destination port */
```

```

    u_int    tcp_seq;                /* sequence number */
    u_int    tcp_ack;                /* acknowledgement number */
    u_char   tcp_offx2;              /* data offset, rsvd */
#define TH_OFF(th)      (((th)->tcp_offx2 & 0xf0) >> 4)
    u_char   tcp_flags;
#define TH_FIN  0x01
#define TH_SYN  0x02
#define TH_RST  0x04
#define TH_PUSH 0x08
#define TH_ACK  0x10
#define TH_URG  0x20
#define TH_ECE  0x40
#define TH_CWR  0x80
#define TH_FLAGS (TH_FIN|TH_SYN|TH_RST|TH_ACK|TH_URG|TH_ECE|TH_CWR)
    u_short  tcp_win;                /* window */
    u_short  tcp_sum;                /* checksum */
    u_short  tcp_urp;                /* urgent pointer */
};

//IP 헤더
void got_packet(u_char *args, const struct pcap_pkthdr *header,
                const u_char *packet)
{
    struct ethheader *eth = (struct ethheader *)packet;

    if (ntohs(eth->ether_type) == 0x0800) { // 0x0800 is IP type
        struct ipheader *ip = (struct ipheader *)
            (packet + sizeof(struct ethheader));

        //MAC 주소를 출력하는 함수
        printf("    MAC Address: %02x:%02x:%02x:%02x:%02x:%02x\n",
            eth->ether_shost[0], eth->ether_shost[1], eth->ether_shost[2],
            eth->ether_shost[3], eth->ether_shost[4], eth->ether_shost[5]);
        printf("    MAC Address: %02x:%02x:%02x:%02x:%02x:%02x\n",
            eth->ether_dhost[0], eth->ether_dhost[1], eth->ether_dhost[2],
            eth->ether_dhost[3], eth->ether_dhost[4], eth->ether_dhost[5]);

        printf("        From: %s\n", inet_ntoa(ip->iph_sourceip));
        printf("        To: %s\n", inet_ntoa(ip->iph_destip));

        /* determine protocol */
        switch(ip->iph_protocol) {
            case IPPROTO_TCP:
                struct tcpheader *tcp = (struct tcpheader *)
                    (packet + sizeof(struct ethheader) + (ip->iph_ihl * 4));
                u_char *http = (u_char *) (packet + sizeof(struct ethheader) + (ip->iph_ihl * 4)
                    + TH_OFF(tcp) * 4);
                printf("    Protocol: TCP\n");
                printf("    Source Port: %d\n", ntohs(tcp->tcp_sport));
                printf("    Destination Port: %d\n", ntohs(tcp->tcp_dport));

                int payload_len = ntohs(ip->iph_len) - (ip->iph_ihl * 4) - (TH_OFF(tcp) * 4);
                printf("    httpmessage: ");
                for (int i = 0; i < payload_len; i++) {
                    unsigned char c = http[i];
                    if (c == '\r' || c == '\n') c = '.';
                    putchar(c);
                }
                putchar('\n');
                return;
            case IPPROTO_UDP:
                printf("    Protocol: UDP\n");
                return;
        }
    }
}

```

```

        case IPPROTO_ICMP:
            printf("    Protocol: ICMP\n");
            return;
        default:
            printf("    Protocol: others\n");
            return;
    }
}

int main()
{
    pcap_t *handle;
    char errbuf[PCAP_ERRBUF_SIZE];
    struct bpf_program fp;
    char filter_exp[] = "tcp";
    bpf_u_int32 net;

    // Step 1: Open live pcap session on NIC with name ens33
    handle = pcap_open_live("ens33", BUFSIZ, 1, 1000, errbuf);

    // Step 2: Compile filter_exp into BPF psuedo-code
    pcap_compile(handle, &fp, filter_exp, 0, net);
    if (pcap_setfilter(handle, &fp) != 0) {
        pcap_perror(handle, "Error:");
        exit(EXIT_FAILURE);
    }

    // Step 3: Capture packets
    pcap_loop(handle, -1, got_packet, NULL);

    pcap_close(handle); //Close the handle
    return 0;
}

```